

Deployment Documentation

Retail Location Intelligence | Café site selection in Çankaya, Ankara

Group 6 · Current main: 58ce550 · Prepared 6 October 2026

1. Overview and actual deployment status

The repository provides a Streamlit/PyDeck application and a Docker configuration for local or self-hosted use. A public cloud URL, production rollout and operating service are not evidenced. This report describes implemented serving behavior and separates it from future production requirements.

2. Model serialization

There is no joblib/pickle model artifact, serialized training pipeline or model registry in the implementation. The persistent artifact is the GeoParquet feature grid, not a saved RF estimator. The app trains grouped RF models at runtime, returns held-out grid scores and caches results with `st.cache_data`. The full-data fit produces descriptive feature importance; it is not serialized. Legacy similarity columns in the stored data are recomputed for display.

3. Model serving and local setup

Serving occurs inside the Streamlit Python process; there is no separate prediction server. Cache inputs include the feature path and modification time, but cache behavior and production invalidation should be reviewed before scale-up. Training failure suppresses old similarity results. The application presents MCDA ranking, RF comparison, filters and competition context for human review.

```
# macOS / Linux: inside the repository
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m streamlit run app.py
```

Open `http://localhost:8501`. Python 3.11 and 3.12 are tested in CI. Installing requirements inside the active environment is necessary, including GeoPandas; a missing-library error does not demonstrate a model error.

4. Docker packaging and health check

```
docker build -t retail-location-intelligence .
docker run --rm -p 8501:8501 retail-location-intelligence
```

The Dockerfile uses `python:3.11-slim`, installs GDAL/GEOS/PROJ packages, exposes port 8501 and runs Streamlit at 0.0.0.0. Its health check requests `/_stcore/health`. Container configuration exists, but a successful image build or hosted-container test is not documented in the available evidence. No latency, concurrency or cost benchmark is claimed.

5. API integration

There is no custom REST prediction endpoint, request schema or response contract. The health endpoint is a Streamlit liveness check, not a prediction API. OSMnx connects to external OSM/geocoding services for collection; TÜİK data is loaded from the repository CSV. An inference API would require a separate trained artifact and validated input schema.

6. Security considerations: implemented versus planned

Area	Current evidence	Production requirement
Authentication / authorization	No application login, user roles or admin-only refresh control is implemented.	Restrict refresh and administrative actions; choose an authentication layer.
Transport / secrets	No app-level TLS configuration or secret-dependent prediction API. Local example uses HTTP.	Use a TLS reverse proxy or hosted platform and secure any future secrets.
Data protection	Public POIs and aggregate residential population; no customer-level records. OSM attribution is required [2].	Review provider terms and privacy before introducing mobility or transaction data.
Write / refresh resilience	Refresh backs up the data directory; failures restore the previous local data and clear caches.	Restrict writes, review concurrency and preserve versioned snapshots.

7. Monitoring, logging and testing

Current monitoring comprises Docker liveness checks, console progress/CV output, Streamlit error messages and GitHub Actions test results. The merged main passed all 43 tests under Python 3.11 and 3.12. There is no structured production log pipeline, automated alerting, drift detector, user analytics or commercial KPI tracker.

Proposed monitoring: track service availability, refresh failures and duration, source snapshot age, missing-feature coverage, training failures, fold ROC-AUC and request latency. Establish baseline values before defining alerts. For the user pilot, record shortlist task time versus the manual workflow and inspect candidate quality. These measures are recommendations, not deployed instrumentation.

8. Operating limitations and conclusion

OSM refresh needs external Overpass/geocoding access. Missing raw layers and incomplete OSM can produce provisional scores; the dashboard communicates gaps. Data refresh changes labels and inputs, so performance must be re-evaluated rather than reusing the recorded 0.960 AUC. Spatial similarity remains separate from commercial success.

The current deliverable is a locally runnable, container-configured decision-support prototype. A production release requires authenticated administration, TLS, reproducible dependencies, versioned data, observability and deployment validation. Model serialization or an API should be added only if the intended serving workflow requires them.

References and implementation evidence

[1] Group 6, current main implementation (Dockerfile, requirements.txt, app.py, .github/workflows/ci.yml, docs/model_validation.md); reviewed 6 October 2026. https://github.com/edasaruhan/SIC_AI_17_Capstone_Group_6/tree/58ce550c027d3ddce9f209fe3979bea849d5716a

[2] OpenStreetMap contributors, copyright and ODbL attribution. <https://www.openstreetmap.org/copyright>